

pico_mst_260219

RP2040 LoRa Master Node Firmware

Project created: 2026-02-19 | Platform: Raspberry Pi Pico (RP2040)

1. Project Overview

This firmware runs on a Raspberry Pi Pico (RP2040 microcontroller) and implements a **LoRa wireless sensor network master node**. The master (MST) bridges a wired UART host application and a mesh of remote LoRa slave nodes. It handles timekeeping via a DS3231 RTC, displays status on an I2C LCD, periodically broadcasts beacon frames, automatically polls slave nodes, and relays host commands to the appropriate slave over the LoRa air link.

The radio module is a **Semtech SX1262** driven by the open-source **RadioLib** library. All firmware is written in C/C++ and built with the **Raspberry Pi Pico SDK 2.2.0** using CMake.

2. Hardware Configuration

2.1 Pin Assignment

Peripheral	Interface	GPIO Pins	Notes
SX1262 LoRa Radio	SPI1	MISO=12, MOSI=11, SCK=10, CS=13	1 MHz SPI clock
SX1262 Control	GPIO	DIO1=20, RST=15, BUSY=2, CS=3	IRQ on rising edge of DIO1
I2C LCD Display	I2C1	SDA=6, SCL=7	400 kHz, I2C address auto-detect
DS3231 RTC	I2C1	SDA=6, SCL=7	Shared I2C bus with LCD
UART Host	UART1	TX=8, RX=9	115200 baud, interrupt-driven RX
Buttons	GPIO	BTN_U=18, BTN_D=19	Defined; not yet active in firmware
Onboard LED	GPIO	PICO_DEFAULT_LED_PIN	1 Hz heartbeat blink

2.2 Radio Parameters

Parameter	Value
Frequency	915.0 MHz
Bandwidth	125.0 kHz
Spreading Factor	7
Coding Rate	5 (4/5)
Sync Word	0x12
TX Power	20 dBm
Mode	Continuous receive; transmit on demand

3. Software Architecture

The firmware uses a **cooperative event-flag state machine** in the main loop. Interrupt service routines (ISRs) set bits in a global flag register `gPARAM_actflag`; the main loop polls this register and dispatches handlers one event per iteration. This avoids preemptive concurrency while keeping ISRs short.

3.1 Event Flags (`gPARAM_actflag`)

Bit Mask	Event	Source	Action
0x01	1-second tick	100 ms repeating timer ($\div 10$)	Toggle LED, refresh LCD second counter
0x02	UART RX complete	UART1 RX ISR	Parse host command, generate response/RF action
0x04	Send UART TX	RF cmd handler or host cmd	Write response buffer to UART1
0x10	Beacon due	Timer: seconds 0, 16, 32	Compose beacon frame, trigger RF TX (0x20)
0x20	RF TX request	Beacon, poll, or host relay	Transmit gPARAM_RFTX_buf via SX1262
0x40	LCD refresh	Timer: every 4 seconds	Update slave data display rows on LCD
0x80	Auto-poll slave	Timer: every 2 seconds	Compose poll for next slave SID, trigger 0x20

3.2 Timer Schedule (1-second counter gPARAM_fcnt_1s)

The 100 ms hardware timer increments a sub-counter; every 10 ticks (1 s) it increments gPARAM_fcnt_1s (0–59) and sets the 1-second event. Additional periodic events are derived from the second counter modulo arithmetic:

Condition	Period	Flag set
fcnt_1s == 0, 16, or 32	≈ 16 s	0x10 (beacon)
fcnt_1s % 4 == 2	4 s	0x40 (LCD refresh)
fcnt_1s % 2 == 1	2 s	0x80 (auto-poll)

3.3 ISR Summary

ISR	Trigger	Action
irqRadio()	GPIO rising edge on DIO1 (pin 20)	Sets receivedFlag = true (if rf_state==0); else resets rf_state and re
on_uart_rx()	UART1 RX FIFO not empty	Assembles byte-stream into gPARAM_UR1RX_buf using length-pr
repeating_timer_callback()	Hardware repeating timer, every 100 ms	Advances sub-counters, sets event flags 0x01 / 0x10 / 0x40 / 0x80

4. Communication Protocol

All packets (both over UART and RF) share the same **12-byte fixed-length frame format**:

Byte(s)	Field	Description
0	hdr / cmd	Command identifier (see table below)
1	length	Always 12 (0x0C)
2	mid	Master ID — fixed 0x02
3	sid	Slave ID: 0x81–0x84, or 0xFF (broadcast)
4	subcmd	Sub-command (0x00=echo, 0xAA=poll, 0x11=write, 0x35=read)
5	addr	Register address (0=time/HH:MM:SS, 1=date, 4=relay/IO)
6–11	data[0..5]	Payload bytes — content depends on cmd/subcmd/addr

Command Codes

hdr (hex)	Direction	Meaning
0x55	MST → SLV (RF)	Beacon (sid=0xFF) or Poll request (sid=target)
0x69	Host → MST (UART)	Write master register (time, date)

0x78	Host → MST (UART)	Write slave register via master relay (e.g. relay control)
0xAA	SLV → MST (RF)	Slave acknowledge / data report
0x89	SLV → MST (RF)	Slave write acknowledgement
0xAA	MST → Host (UART)	Echo / acknowledgement response
0x7A	MST → Host (UART)	Master register read-back response

Auto-Poll Slave List

The master cycles through up to 4 slave IDs: **0x81, 0x82, 0x83, 0x84**. Every 2 seconds it polls the next slave in the list, wrapping at gPARAM_A POLL_cnt_lmt (default 3). Slave responses are parsed by mst_rf_cmd_process() and displayed on the LCD.

5. Source File Structure

File	Language	Purpose
pico_mst_260219.cpp	C++	Entry point: hardware init, ISRs, event-loop state machine
global_def.h	C/C++	GPIO pin defines, packed structs (TimeData, BcnData, PollData, slv_Data, dec3/5Dig)
main.h	C/C++	Global variable definitions (all buffers, counters, auto-poll config)
mst_ur_proc.cpp / .h	C++	UART command parser and packet composers (beacon, poll, write relay, responses)
mst_rf_cmd_proc.cpp / .h	C++	RF receive parser; UART response composer; LCD display buffer fill
usr_lib/i2c_lcd.cpp / .h	C++	I2C LCD driver (16×4 character display)
usr_lib/ds3231.c / .h	C	DS3231 RTC driver (time read/write, EEPROM)
RadioLib/	C++	Third-party SX126x radio library (vendored, Sep 2025)
CMakeLists.txt	CMake	Build definition: SDK 2.2.0, toolchain 14.2, RADIOLIB_BUILD_RPI_PICO
pico_sdk_import.cmake	CMake	Pico SDK import helper

6. Build Configuration

Setting	Value
Target board	pico (RP2040)
Pico SDK version	2.2.0
Toolchain	ARM GCC 14.2 Rel1
C standard	C11
C++ standard	C++17
stdio via USB	Enabled (pico_enable_stdio_usb)
stdio via UART	Disabled
RadioLib compile def	RADIOLIB_BUILD_RPI_PICO
Linked libraries	pico_stdlib, hardware_spi, hardware_i2c, hardware_timer, hardware_uart, hardware_pwm, pico_

7. Known Issues & Bugs

The following defects were identified during code analysis:

[BUG — compile error] pico_mst_260219.cpp:106 — conv4byte_4decdig() references undefined variable intval; should be the parameter inval. The file will not compile.

[BUG — logic error] pico_mst_260219.cpp:290 — if(gPARAM_APOLL_en = 1) uses assignment instead of comparison (==). Auto-poll is always enabled regardless of the variable's value.

[BUG — wrong field] mst_ur_proc.cpp:139 — mst_ur_rsp_write_mst_a1() writes Stru_curTime.day to both byte 8 and byte 9. Byte 9 should carry Stru_curTime.dow (day-of-week).

[BUG — unconditional display fill] mst_rf_cmd_proc.cpp:46–55 — The LCD display buffer is always filled from inbuf even when the RF packet failed to parse. Garbage data may appear on the LCD after a bad receive.

[BUG — auto-poll counter wrap] pico_mst_260219.cpp:292–296 — When cnt >= lmt, the counter is reset to 0 without incrementing, so the same slave (index 0) is polled twice in a row after each wrap.

[INFO — inefficient math] pico_mst_260219.cpp:167–226 — conv1byte_3dig() performs decimal digit extraction by repeated subtraction instead of using % and / operators. Functionally correct but slow.